Algebraic Data Types

Learning Outcomes

by the **End** of the **Lecture**, **Students** that **Complete** the **Recommended Exercises** should be **Able** to:

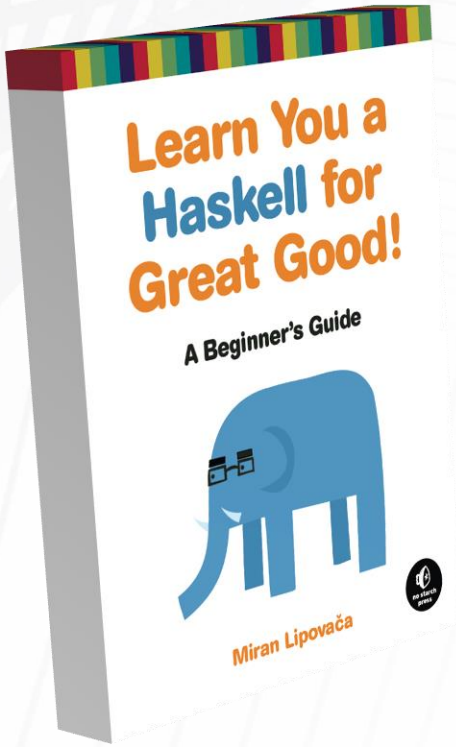
Describe Algebraic Data Types

Use Typeclass Derivation

Use Option Types, Maybe, Just, and Nothing

Define a Recursive Data Type for a Implementing a Tree

Reading and Suggested Exercises



Read the following **Chapter(s)**:

Chapter 8:
"Algebraic data types intro"
"Type parameters"
"Derived instances"

Algebraic Data Types

Lists are **Special Instances** of the **More General Algebraic Data Type**

as noted previously, there are **Two Types** of **List**: **Empty Lists** and **Cons Pairs**

Algebraic Data Types

this entails that **If** there was **No List Type**,
then a **Special Instance** of a **Algebraic Data Type**
for storing a **List** of **Integers** could be defined:

```
data ListInts = EmptyList | (ConsPair Integer ListInts)
```

How could the **List** `[1, 2, 3]` be **Stored**
Using this **Custom Type**?

Algebraic Data Types

remember that "**:**" is an **Infix Binary Operator** used to **Associate** a **Head Element** with a **Tail List** for the **Recursive Definition** of **List**

```
[1, 2, 3]
= (1 : [2, 3])
= (1: (2 : [3]))
= (1 : (2 : (3 : [])))
```

Algebraic Data Types

```
data ListInts = EmptyList | (ConsPair Integer ListInts)
```

Using this **Special Type**, `(1 : (2 : (3 : [])))` could be **Written** as:

```
(ConsPair 1 (ConsPair 2 (ConsPair 3 EmptyList)))
```

Algebraic Data Types

```
data ListInts = EmptyList | (ConsPair Integer ListInts)
```

How would the `sum` and `concat` **Functions Implemented Without** the **Tail Call Optimization**) be **Redefined** for **ListInts**?

Algebraic Data Types

```
data ListInts = EmptyList | (ConsPair Integer ListInts)
```

How would you **Redefine** the `ListInts` **Type** to be **Generic** (i.e., not only for integers)?

What Changes would be **Required** to **Allow** `sum` and `concat` to work with the **Generic List Type**?

Algebraic Data Types

```
data ListInts
= EmptyList | ConsPair Int ListInts

sumList :: ListInts -> Int
sumList EmptyList = 0
sumList (ConsPair h t) = h + sumList t

concatList :: ListInts -> ListInts -> ListInts
concatList EmptyList x = x
concatList (ConsPair h t) x =
                          ConsPair h (concatList t x)
```

Algebraic Data Types

```
data List a
= EmptyList | ConsPair a (List a)

sumList :: List a -> a
sumList EmptyList = 0
sumList (ConsPair h t) = h + sumList t

concatList :: List a -> List a -> List a
concatList EmptyList x = x
concatList (ConsPair h t) x =
                          ConsPair h (concatList t x)
```

Simple Applications for **Algebraic Data Types** include **Enumerated** or **Categorical Types**

```
data Adjective
      = Excellent | Good | Satisfactory | Poor

data Letter
      =     A | B | C | D | F

autoComment :: Letter -> Adjective
autoComment A = Excellent
autoComment B = Good
autoComment C = Satisfactory
autoComment _ = Poor
```

Algebraic Data Types

```
data BooleanValue = MyFalse | MyTrue
```

With this **Algebraic Data Type** for **Boolean Values**, **How** (using **Pattern Matching**) could you **Implement** the **Conjunction** and **Disjunction Operators**?

```
logicalAND :: BooleanValue -> BooleanValue -> BooleanValue
```

```
logicalIOR :: BooleanValue -> BooleanValue -> BooleanValue
```

Algebraic Data Types

```
data BooleanValue = MyFalse | MyTrue

logicalAND MyFalse _ = MyFalse
logicalAND _ MyFalse = MyFalse
logicalAND _ _ = MyTrue

logicalIOR MyTrue MyTrue = MyTrue
logicalIOR MyTrue _ = MyTrue
logicalIOR _ MyTrue = MyTrue
logicalIOR _ _ = MyFalse
```

Can the **Same Functionality** be **Accomplished** using **Fewer Patterns**?

Algebraic Data Types

What was the **Nature** of the **Problem Encountered** during **Testing**?

Typeclass Derivation

```
showBooleanValue :: BooleanValue -> String
showBooleanValue MyTrue = "true"
showBooleanValue MyFalse = "false"
```

fortunately there is a **Simpler Approach** for **Ensuring** that a **Result** can be "**Shown**" ... using **Derivation**

Typeclass Derivation

```
data BooleanValue = MyFalse | MyTrue deriving (Show)

logicalAND MyTrue MyTrue = MyTrue
logicalAND _ _ = MyFalse

logicalIOR MyFalse MyFalse = MyFalse
logicalIOR _ _ = MyTrue
```

now **What Happens** during `logicalAND MyFalse MyTrue` ?

Typeclass Derivation

the deriving **Keyword** allows an **Algebraic Data Type** to become an **Instances** of a **Typeclass**

this **Grants "Special" Functionality Automatically**

Typeclasses like `Show`, `Eq`, `Ord` are for **Types** that can **Naturally** be **Shown**, **Equated**, and **Ordered**, respectively

Algebraic Data Types

Each of the **List Types** has its own **Constructor**

the **Empty List** is **Associated** with **Constructor** "`[]`" and the **Cons Pair Variant** (that **Combines** a **Head** and a **Tail**) is **Associated** with the **Constructor** "`:`"

How many **Values** are **Required** by **Each** of these **Constructors**?

Algebraic Data Types

suppose you were providing **Functionality** for **Computing Properties** of **Circles** and **Rectangles**?

conceptually **Circles** and **Rectangles** would both be **Instances** of a **Shape Class**, so **What Algebraic Data Type** could you **Define**?

What are the **Constructors** and
What does **Each Constructor Require**?

Algebraic Data Types

a **Function** for **Creating** a **Rectangle** would **Require** a **Width** and a **Height**, while a **Function** for **Creating** a **Circle** would only **Require** a **Radius**

```
data Shape =
Rectangle Float Float | Circle Float
```

Algebraic Data Types

```
data Shape =
Rectangle Float Float | Circle Float
```

What is the **Type Definition** for a **Function** that **Computes** the **Area** of a **Circle** (but **Not** a **Rectangle**) ?

How would you **Implement** this **Function**?

Algebraic Data Types

```
data Shape =
Rectangle Float Float | Circle Float

area :: Shape -> Float
area (Circle r) = pi * r * r
```

Why would `area :: Circle -> Float` be **Incorrect**?

Why would `area (Shape r) = pi * r * r` be **Incorrect**?

How would you **Add** the **Functionality** for **Rectangular Areas** and **Circular** and **Rectangular Perimeters**?

Algebraic Data Types

recalling that the `:t` and `:type` **Commands** in **GHCi** will **Provide** the **Type Definition**, **What** is the **Response** from **GHCi** to **Each** of the **Following**?

```
:t area

:t perimeter

:t Circle

:t Rectangle
```

Option Types

when **Multiplying Integers** (i.e., the **Argument Type** is **Integer**), what is the **Type** of the **Product**?

when **Multiplying Floats** (i.e., the **Argument Type** is **Float**), what is the **Type** of the **Product**?

when **Dividing Integers** (i.e., the **Argument Type** is **Integer**), what is the **Type** of the **Quotient**?

when **Dividing Floats** (i.e., the **Argument Type** is **Float**), what is the **Type** of the **Quotient**?

Option Types

there are **Some Situations** (i.e., **Expressions**)
where the **Result Type** is **Not Definite**

e.g., an **Operator** that **Produces** a **Number** in
Certain Cases but **Something Else** in **Other Cases**

Option Types

with the **Understanding** that **Division** (applied to floating point numbers) can **Result** in **Either** a **Float** or a **Special Value** in cases where the **Divisor** is 0 (usually called "**Undefined**")...

...**How** would you **Define** a **Custom Type** to **Handle Possible Quotients**?

Option Types

```
data QuotientType =
    UndefinedType | FloatType Float

floatDivision :: Float -> Float -> QuotientType
floatDivision _ 0 = UndefinedType
floatDivision x y = FloatType (x / y)

showQuotientType :: QuotientType -> String
showQuotientType UndefinedType = "Don't Divide by Zero!"
showQuotientType (FloatType z) = show z
```

Option Types

```
data QuotientType =
UndefinedType | FloatType Float deriving (Show)

floatDivision :: Float -> Float -> QuotientType
      floatDivision _ 0 = UndefinedType
    floatDivision x y = FloatType (x / y)
```

Option Types

in fact it is **Not Necessary** to **Develop Custom Algebraic Data Types** for every **Operator** that exhibits this kind of **Behaviour**...

...for these cases, use **Maybe**, **Just**, and **Nothing**

Maybe, Just, and Nothing

the `Maybe` **Data Type** is an **Enumerated Type**

```
data Maybe a = Just a | Nothing deriving (Eq, Ord)
```

this allows you to potentially **Work With** a **Value** that **Might Not Exist** or **Might Not Be Available**

if the **Value Exists** and is **Available**, it is "**Wrapped**" in the `Just` **Constructor**, and **Otherwise** it is `Nothing`

Maybe, Just, and Nothing

Using Maybe, **How** would you **Implement** a **Function** that can **Safely Return** the **2nd Element** of a **List**?

n.b., in this context, safely means only making the attempt if you know the element exists

Maybe, Just, and Nothing

```
import Data.Maybe

safelyGet2nd :: [a] -> (Maybe a)

safelyGet2nd [] = Nothing

safelyGet2nd (h:t)
  | length t == 0 = Nothing
  | otherwise = Just (head t)
```

Maybe, Just, and Nothing

an **Alternative Approach** is to **Use Monads**

```
safelyGet2nd' x = case x of
  (h:(a:b)) -> Just a
  _ -> Nothing
```

Monads are used to **Represent Sequences** of **Computations** (allowing for things like **User Input** that would **Violate Referential Transparency**)

n.b., due to the complications that can arise, monads are beyond the scope of this course

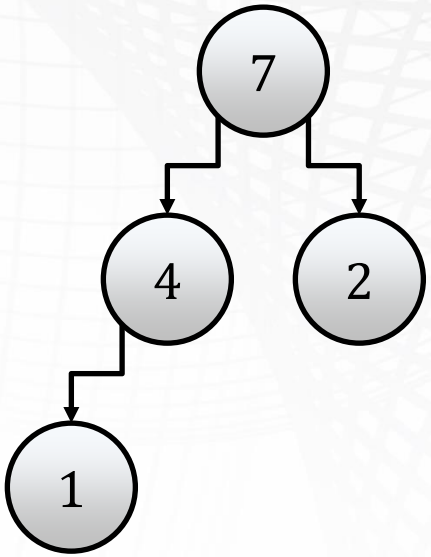
Recursive Data Structures

Algebraic Data Types can also be **Used** to **Define Other Data Types Recursively**

What are the **Possible Variants** of a **Binary Tree** of **Integers** (i.e., **Every Internal** node contains an **Integer** and has **No More** than **Two Child** nodes)?

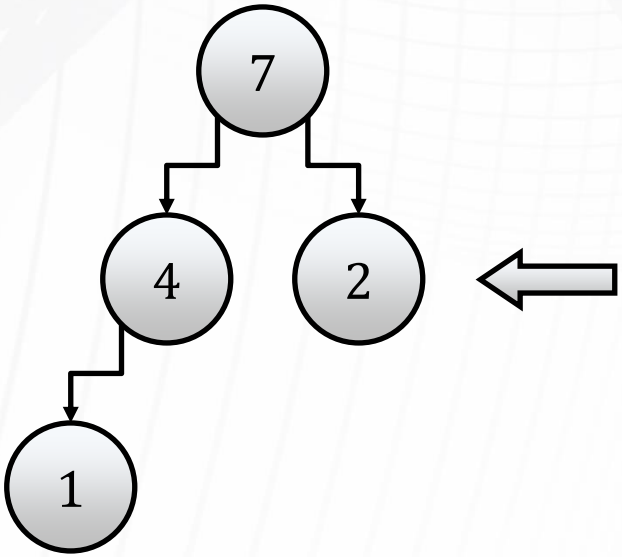
n.b., you must decide on value constructor labels and the arguments necessary for each

Recursive Data Structures



How would you **Encode** this **Tree**?

Recursive Data Structures




```
data NTree
= NilT | Node Int Ntree Ntree
```

```
Node 7
     (Node 4
          (Node 1 NilT NilT)
          NilT)
     (Node 2 NilT NilT)
```

Recursive Data Structures

Using a **Style** similar to that used for **Recursive List Processing**, **How** would **Write** a **Function** to **Compute** the **Sum** of the **Elements** in such a **Tree**?

Recursive Data Structures

```
data NTree = NilT | Node Int NTree NTree

sumTree :: NTree -> Int

sumTree NilT = 0
sumTree (Node n lt rt) = n + (sumTree lt) + (sumTree rt)
```

How would you **Trace** the **Following**?

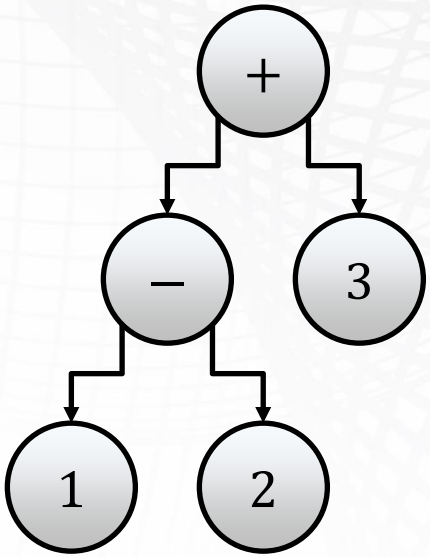
```
sumTree (Node 7 (Node 4 (Node 1 NilT NilT) NilT) (Node 2 NilT NilT))
```

What about the **Trees Used** to **Represent Arithmetic Expressions**?

```
data Expr =
Lit Int | Add Expr Expr | Sub Expr Expr | Mul Expr Expr
```

How would you **Represent** $(1 - 2) + 3$?

Recursive Data Structures



How would you **Write Functions** to **Evaluate Expressions Encoded** with this **Structure**?

n.b., to evaluate an expression tree you evaluate subtrees an perform operations